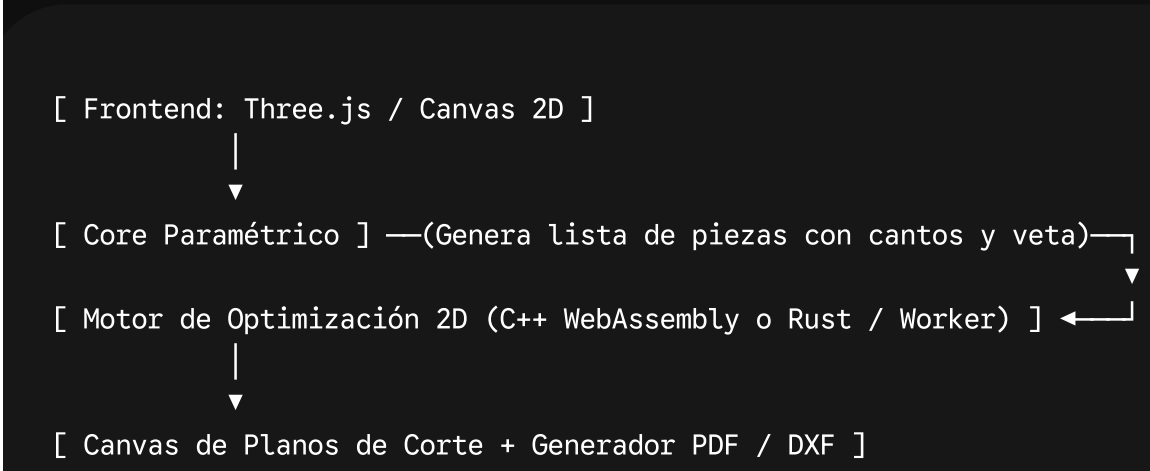
crear clon de melastudio pro

Para construir un clon o una alternativa propia a una suite de diseño de muebles, despiece y optimización de tableros de melamina (como **MelaStudio / MuebleMIO / Cutting Optimization Pro**), el sistema se divide en cuatro módulos esenciales: modelado paramétrico 3D, generador de despiece con reglas de cantos, algoritmo de anidamiento 2D (*nesting/bin-packing*) y exportación de reportes.

1. Arquitectura del Sistema



2. Stack Tecnológico Recomendado

- **Frontend / UI: Angular** o **React** con Tailwind CSS.
- **Visualizador 3D (Mueble interactivo): Three.js** o `@react-three/fiber` para renderizar módulos (gabinetes, cajones, repisas) con dimensiones dinámicas.
- **Motor de Optimización (2D Bin Packing):** Desarrollado en **Rust/C++ compilado a WebAssembly** o ejecutado en un **Web Worker** en JavaScript/TypeScript para no congelar la interfaz.
- **Canvas de Corte 2D:** HTML5 Canvas o Konva.js para dibujar tableros, líneas de sierra y cotas de corte.
- **Reportes / Impresión:** `jsPDF` y `jspdf-autotable` para generar hojas de corte listas para taller o centros de servicio (Sodimac, Placacentro, Pelíkano, etc.).

3. Módulos Esenciales a Desarrollar

A. Modelo de Datos y Parametrización

Cada mueble se compone de partes hijas cuyas cotas dependen de las variables maestras: `Ancho`, `Alto`, `Profundidad`, `Espesor del tablero` (ej. 15mm, 18mm).

TypeScript

```typescript
export interface PiezaDespiece {
  id: string;
  nombre: string;              // ej. "Lateral Izquierdo", "Base"
  largo: number;               // mm
  ancho: number;               // mm
  cantidad: number;
  espesor: number;             // 15, 18, 36 mm
  veta: 'longitudinal' | 'transversal' | 'libre'; // Restricción de giro
  cantos: {
    largo1: number;            // 0 = sin canto, 0.45 = delgado, 1 = medio, 3 = grueso
    largo2: number;
    ancho1: number;
    ancho2: number;
  };
  ranuras?: { eje: 'X' | 'Y'; distancia: number; ancho: number; profundidad: number }
}
```

> **Regla de taller:** Al aplicar cantos gruesos (ej. 2mm o 3mm), el corte neto del tablero debe descontar el grosor del tapacanto antes de enviarlo al optimizador.

B. Motor de Optimización de Corte (2D Guillotine Bin Packing)

A diferencia del corte láser/router CNC (nesting libre con piezas curvas), las seccionadoras y sierras de banco de melamina requieren **cortes de guillotina** (cortes rectos de lado a lado).

1. **Parámetros de entrada de la plancha:**
   Cotcutly | Despiece y Pre...
   1. Dimensiones de la lámina (ej. 2140×2440 mm o 1830×2600 mm).
   2. Refilado perimetral (*trimming*): 10 mm a 15 mm en cada borde para eliminar esquinas golpeadas.
   3. Espesor de la hoja de sierra (*kerf*): 4 mm a 4.5 mm por cada pasada de corte.
2. **Algoritmo principal:**
   1. Utiliza variantes de **Guillotine Bin Packing** con heurísticas como *Best Area Fit (BAF)*, *Best Short Side Fit (BSSF)* o *Maximal Rectangles (MaxRects)*

adaptado a cortes de guillotina.

2. Si la pieza tiene veta fija, se prohíbe la rotación de 90°.
3. Evalúa fases de corte: primero cortes longitudinales (rasgar la plancha) o primero transversales.

C. Integración con SketchUp / OpenCutList (Interoperabilidad)

Para permitir que usuarios diseñen en SketchUp y procesen cortes en tu aplicación:

- Crea un parser que acepte archivos **CSV / Excel** generados por plugins como **OpenCutList (OCL)**.
- El parser debe mapear columnas: `Largo`, `Ancho`, `Espesor`, `Cantidad`, `Material`, y marcas de cantos.

4. Roadmap para un Producto Mínimo Viable (MVP)

1. **Fase 1 (Optimizador Web 2D):**
    1. Tabla interactiva para ingresar dimensiones, cantidad, veta y espesor de sierra.
    2. Algoritmo guillotine 2D con render en `<canvas>` numerando la secuencia de cortes.
    3. Exportación de resumen: porcentaje de desperdicio, número de tableros y metros lineales de canto.
2. **Fase 2 (Generador de Muebles Básicos):**
    1. Formularios paramétricos preconfigurados (módulo bajo de cocina, módulo aéreo, repisero, clóset básico).
    2. Cálculo automático de despiece con un clic.
3. **Fase 3 (Visor 3D y Presupuestos):**
    1. Vista 3D interactiva en WebGL/Three.js para verificar ensamble y orientación de vetas.
    2. Calculadora de costos: precio por plancha, costo por metro de tapacanto y bisagras/correderas.

aqui tengo una demostracion del pluging saca todas las funcionalidades del pluging bondades, beneficios, https://www.youtube.com/watch?v=ZfgfMiF0LPg&t=76s

A partir del análisis del video El Aplicativo que Todo Fabricante de Muebles de Melamina Debería Conocer, se detallan todas las funcionalidades, bondades y beneficios de este plugin para diseño y despiece automatizado en SketchUp:

1. Funcionalidades Técnicas del Plugin

- **Generación paramétrica instantánea de módulos:** En lugar de dibujar pieza por pieza en SketchUp (como se hace tradicionalmente o con OpenCutList dibujando rectángulos y extruyendo a 18 mm), el plugin cuenta con un formulario donde introduces:
  - **Nombre o tipo de estructura:** Por ejemplo, librero o cubo [02:24].
  - **Cotas generales:** Ancho, alto y profundidad en milímetros [02:31].
  - **Accesorios estructurales:** Opción directa de activar/desactivar ranurado (*ranura para fondo*) [02:48].
- **Configuración del espacio interior y repisas:** Permite definir la distribución interna (ej. agregar repisas o divisiones con un clic y fijar cotas de altura para cada nicho, como dejar 300 mm en la parte inferior) [02:59], [06:22].
- **Edición paramétrica no destructiva:** Para modificar un mueble ya creado, no se mueven las caras manualmente en SketchUp; basta con abrir el módulo del plugin, cambiar los parámetros numéricos y darle a actualizar/crear [06:16].
- **Extracción automatizada de lista de despiece:** Incluye un botón para extraer la lista completa y exportarla directamente a un archivo de Excel listo para producción [04:40], [04:52].
- **Cálculo de componentes especiales:** Calcula de forma independiente el fondo de MDF de 3 mm ajustado a la ranura, además de las piezas estructurales de melamina [05:27].
- **Lógica de cantos y descuentos:** Registra el tipo de canto (delgado/grueso) y aplica los descuentos milimétricos necesarios para obtener la medida neta de corte de taller [05:12].

2. Bondades (Características Diferenciales frente al Flujo Tradicional)

Flujo Tradicional (SketchUp + OCL)	Flujo con el Plugin
Dibujar rectángulos, dar espesor (18 mm), crear grupos/componentes [01:09].	Configurar dimensiones en una ventana y hacer clic en **Crear** [02:59].

Flujo Tradicional (SketchUp + OCL)	Flujo con el Plugin
Copiar, mover y medir manualmente cada repisa y lateral [03:49].	El motor ensambla automáticamente laterales, techo, piso y divisiones.
Modificar requiere estirar piezas, reubicar repisas y verificar cotas a mano.	Modificación por formulario paramétrico en segundos [06:31].
Riesgo de olvidar descontar el ranurado del fondo o el espesor de tapacantos.	Descuentos de cantos y cálculo de ranura integrados por defecto [05:20].

...

3. Beneficios Directos para el Fabricante y Diseñador

- **Ahorro masivo de tiempo (Diseño en tiempo récord):** Reduce el proceso de modelado de varios minutos u horas por mueble a solo unos cuantos clics y segundos [06:49].
- **Eliminación del error humano en taller:** Al automatizar el despiece directamente desde los parámetros, se evitan errores de cálculo manual en piezas duplicadas o descuentos de cantos [05:12].
- **Listo para centros de corte:** La salida en Excel ya cuenta con el formato estándar (medida neta, cantidad, indicación de ranuras y tapacantos) listo para ser ingresado a sistemas de corte de tiendas y distribuidoras (Pelíkano, Masisa, Sodimac, Promart, etc.) [05:20].
- **Curva de aprendizaje reducida:** No exige un dominio avanzado de herramientas nativas de modelado 3D en SketchUp; cualquier operario o carpintero puede generar muebles estándar introduciendo medidas [07:05].

Consideraciones para tu Clon

Si vas a replicar esta solución (sea como extensión de SketchUp en Ruby/HTML Dialogs o como aplicación web independiente):

1. **Estructura de datos tipo *Box/Cabinet*:** Define clases base para módulos predeterminados (`Cubo` , `ModuloBajo` , `ModuloAlto` , `Cajonera`).
2. **Motor de descuentos:** Cada pieza secundaria (`techo` , `piso` , `repisas`) debe derivar sus medidas automáticamente:

$$\text{Ancho repisa} = \text{Ancho total} - (2 \times \text{Espesor lateral})$$

3. **Módulo de ranura y fondo:**

$$\text{Medida fondo} = (\text{Espacio interior} + 2 \times \text{Profundidad de ranura}$$

4. **Exportador CSV/XLSX:** Formatea la salida agrupando piezas por material (Melamina 18 mm vs. MDF 3 mm) y asignando las cuatro marcas de canto por lado (`L1` , `L2` , `A1` , `A2`).

El Aplicativo que Todo Fabricante de Muebles de Melamina Debería Conocer
DFI Creativo · 456 vistas

Abrir en El Aplicativo que Todo Fabricante de Muebles de Melamina Debería Conocer



mencionaste el desarrollo en angular y ahora en ruby, cual se va a utilizar

Depende exclusivamente de **dónde quieres que viva tu aplicación**: si dentro de **SketchUp** (como el video que mostraste) o como una **Plataforma Web independiente** en el navegador.

Opción 1: Plugin para SketchUp (Usa Ruby + HTML/JS)

Si tu meta es hacer exactamente lo que se ve en el video de demostración:

- **Por qué Ruby:** La API oficial de extensiones de SketchUp está escrita en **Ruby**. Cualquier acción que cree geometrías 3D nativas, grupos, componentes o extraiga piezas dentro de SketchUp se ejecuta obligatoriamente en Ruby.

- **Dónde entra el frontend:** La ventana flotante donde el usuario escribe el alto, ancho y profundidad es un `UI::HtmlDialog` de SketchUp. Esa interfaz interna se programa con HTML, CSS y JavaScript (puedes usar JS puro, o incluso empaquetar una vista ligera).
- **Flujo:** El usuario llena el formulario en HTML/JS $\rightarrow$ se envía un callback a Ruby $\rightarrow$ Ruby dibuja los paneles de melamina dentro de la escena de SketchUp y exporta el Excel.

Opción 2: Aplicación Web Independiente / SaaS (Usa Angular + WebGL/Three.js)

Si tu meta es crear un sistema tipo **MelaStudio / MuebleMIO web**, accesible desde cualquier navegador sin obligar al usuario a tener instalado ni pagar licencias de SketchUp:

- **Por qué Angular:** Es el framework frontend donde vive toda la aplicación: la interfaz de usuario, los formularios de módulos, la gestión de proyectos y la tabla interactiva de despiece.
- **Visualización 3D:** En lugar de SketchUp, usas una librería de WebGL como **Three.js** incrustada en un componente de Angular para renderizar el mueble en 3D en tiempo real.
- **Optimización y exportación:** Se ejecuta en el cliente (mediante Web Workers o WebAssembly) o a través de un backend (Node.js/NestJS) para procesar el corte 2D y generar los PDFs o archivos para centros de corte.

Comparativa directa

Criterio	Plugin de SketchUp (Ruby)	Aplicación Web (Angular + Three.js)
Ecosistema	Requiere SketchUp instalado en la PC.	100% en el navegador (PC, tablet o móvil).
Público objetivo	Diseñadores y carpinteros habituados a SketchUp.	Clientes finales, talleres pequeños y vendedores de tiendas de corte.
Complejidad de desarrollo 3D	**Baja-Media:** SketchUp ya resuelve la cámara, luces, texturas y render. Solo envías comandos de dibujo en Ruby.	**Media-Alta:** Debes programar el visor 3D, controles de cámara y materiales desde cero en Three.js.

Criterio	Plugin de SketchUp (Ruby)	Aplicación Web (Angular + Three.js)
Distribución / Negocio	Archivo `.rbz` que se instala localmente.	Software como Servicio (SaaS), control total de suscripciones y accesos.

...

Veredicto para tu proyecto

- Si quieres **reproducir fielmente el plugin del video** para venderlo o usarlo dentro del flujo de trabajo existente de SketchUp: **Usa Ruby con una UI en HTML/JS**.
- Si buscas **crear un producto propio independiente**, que no dependa de Trimble/SketchUp y donde el usuario diseñe, optimice y saque la lista de corte desde una web: **Usa Angular + Three.js**.

me voy por la opcion 1, acuerdate que las medidas deben ser milimetricas un error puede ocasionar un mal despiece

Para trabajar con precisión milimétrica dentro de la API de Ruby de SketchUp, hay una regla de oro fundamental: **SketchUp maneja todas sus dimensiones internas en pulgadas (*inches*)**.

Si no conviertes explícitamente las medidas con el método `.mm` de la API de SketchUp, se producirán desfases que arruinarán el despiece. Al usar `valor.mm`, Ruby transforma automáticamente los milímetros exactos al sistema interno de coordenadas sin errores de punto flotante.

A continuación tienes la arquitectura base, la estructura de archivos y el código para inicializar tu extensión.

1. Estructura de Archivos del Plugin

En tu carpeta de plugins de SketchUp (o en tu entorno de desarrollo para empaquetar el `.rbz`):

Plaintext

```
mela_studio_clone/
│
├── mela_studio_clone.rb            # Registrador de la extensión en SketchUp
└── mela_studio_clone/
    ├── main.rb                     # Lógica principal, menú y listeners de Ruby
    ├── cabinet_builder.rb          # Generador geométrico y matemático milimétrico
    └── web/
        ├── index.html              # UI (HtmlDialog)
        ├── style.css               # Estilos de la ventana
        └── app.js                  # Captura de inputs y comunicación con Ruby
```

2. Registrador de la Extensión (`mela_studio_clone.rb`)

Este archivo se coloca directamente en la raíz de plugins de SketchUp para registrar la herramienta:

Ruby

```ruby
require 'sketchup.rb'
require 'extensions.rb'

module MelaStudioClone
  unless file_loaded?(__FILE__)
    ex = SketchupExtension.new('MelaStudio Clone', 'mela_studio_clone/main')
    ex.description = 'Diseño paramétrico y despiece milimétrico automatizado de mueble
    ex.version     = '1.0.0'
    ex.creator     = 'Tu Nombre / Taller'
    Sketchup.register_extension(ex, true)
    file_loaded(__FILE__)
  end
end
```

3. Lógica del Menú e Interfaz (`mela_studio_clone/main.rb`)

Abre el cuadro de diálogo HTML nativo (`UI::HtmlDialog`) y establece el puente de comunicación bidireccional entre JavaScript y Ruby:

Ruby

```ruby
require 'sketchup.rb'
require 'json'
require_relative 'cabinet_builder'

module MelaStudioClone
  class App
    def self.show_dialog
      @dialog ||= UI::HtmlDialog.new({
        :dialog_title    => "MelaStudio - Generador Paramétrico",
        :preferences_key => "com.melastudio.paramdialog",
        :scrollable      => true,
        :resizable       => true,
        :width           => 420,
        :height          => 600,
        :style           => UI::HtmlDialog::STYLE_DIALOG
      })

      html_path = File.join(__dir__, 'web', 'index.html')
      @dialog.set_file(html_path)

      # Callback que recibe los datos desde JavaScript
      @dialog.add_action_callback("crearModulo") do |action_context, params_json|
        data = JSON.parse(params_json)
        CabinetBuilder.build(data)
      end

      @dialog.show
    end
  end

  unless file_loaded?(__FILE__)
    menu = UI.menu('Plugins')
    menu.add_item('Abrir MelaStudio Paramétrico') { App.show_dialog }
    file_loaded(__FILE__)
  end
end
```

4. Motor Matemático y Geométrico Milimétrico (`mela_studio_clone/cabinet_builder.rb`)

Aquí reside la precisión del despiece. Este archivo:

1. Convierte las cotas a milímetros (`.mm`).

2. Aplica descuentos estructurales (laterales externos cubriendo techo y piso).
3. Etiqueta cada pieza en un `AttributeDictionary` nativo de SketchUp para que el extractor de despiece pueda leer los nombres, cantos y medidas reales.

Ruby

```ruby
module MelaStudioClone
  class CabinetBuilder
    def self.build(params)
      model = Sketchup.active_model
      model.start_operation('Crear Módulo Melamina', true)

      # 1. Conversión estricta a milímetros usando la API nativa
      ancho_total = params['ancho'].to_f.mm
      alto_total  = params['alto'].to_f.mm
      prof_total  = params['profundidad'].to_f.mm
      t           = params['espesor'].to_f.mm # 18.mm o 15.mm

      tiene_fondo = params['fondo'] == true
      prof_ranura = tiene_fondo ? 8.mm : 0.mm # Profundidad de ranura en laterales

      entities = model.active_entities
      grupo_modulo = entities.add_group
      grupo_modulo.name = params['nombre'] || "Módulo Base"

      # 2. Laterales (Izquierdo y Derecho)
      # Medida bruta: Alto total x Profundidad total
      crear_panel(grupo_modulo, "Lateral Izquierdo",
                  0.mm, 0.mm, 0.mm,
                  t, prof_total, alto_total,
                  { "Largo" => params['alto'], "Ancho" => params['profundidad'], "Esp

      crear_panel(grupo_modulo, "Lateral Derecho",
                  ancho_total - t, 0.mm, 0.mm,
                  t, prof_total, alto_total,
                  { "Largo" => params['alto'], "Ancho" => params['profundidad'], "Esp

      # 3. Piso y Techo (Van entre laterales: Ancho = Ancho_total - 2*espesor)
      ancho_interno = ancho_total - (2 * t)
      ancho_interno_num = params['ancho'].to_f - (2 * params['espesor'].to_f)

      crear_panel(grupo_modulo, "Piso",
                  t, 0.mm, 0.mm,
                  ancho_interno, prof_total, t,
                  { "Largo" => ancho_interno_num, "Ancho" => params['profundidad'], "
```

```
    crear_panel(grupo_modulo, "Techo",
                t, 0.mm, alto_total - t,
                ancho_interno, prof_total, t,
                { "Largo" => ancho_interno_num, "Ancho" => params['profundidad'], "

    # 4. Repisa Intermedia (si se especifica)
    if params['tiene_repisa']
      altura_repisa = params['altura_repisa'].to_f.mm
      crear_panel(grupo_modulo, "Repisa",
                  t, 0.mm, altura_repisa,
                  ancho_interno, prof_total, t,
                  { "Largo" => ancho_interno_num, "Ancho" => params['profundidad'],
    end

    # 5. Fondo MDF 3mm (si tiene ranura)
    if tiene_fondo
      fondo_ancho = ancho_interno + (2 * prof_ranura)
      fondo_alto  = (alto_total - (2 * t)) + (2 * prof_ranura)
      t_fondo     = 3.mm
      # Ubicado a 18mm desde atrás
      crear_panel(grupo_modulo, "Fondo MDF 3mm",
                  t - prof_ranura, prof_total - 18.mm - t_fondo, t - prof_ranura,
                  fondo_ancho, t_fondo, fondo_alto,
                  { "Largo" => fondo_alto.to_mm, "Ancho" => fondo_ancho.to_mm, "Esp
    end

    model.commit_operation
  end

  # Dibuja un paralelepípedo limpio y guarda sus metadatos exactos de despiece
  def self.crear_panel(parent_group, nombre, x, y, z, dx, dy, dz, despiece_attrs)
    subgrupo = parent_group.entities.add_group
    subgrupo.name = nombre

    # Crear cara base
    pts = [
      Geom::Point3d.new(x, y, z),
      Geom::Point3d.new(x + dx, y, z),
      Geom::Point3d.new(x + dx, y + dy, z),
      Geom::Point3d.new(x, y + dy, z)
    ]
    face = subgrupo.entities.add_face(pts)
    face.pushpull(dz)

    # Almacenar datos en AttributeDictionary para exportación milimétrica a Excel/C
    dict = subgrupo.attribute_dictionary("DespieceMela", true)
    despiece_attrs.each do |k, v|
      dict[k] = v.to_s
    end
```

```
    end
  end
end
```

5. Interfaz HTML/JS (`mela_studio_clone/web/index.html`)

Formulario minimalista y directo para ingresar las cotas:

HTML

```html
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <style>
    body { font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, sans-s
    .form-group { margin-bottom: 12px; }
    label { display: block; font-weight: 600; margin-bottom: 4px; color: #333; }
    input[type="number"], input[type="text"], select { width: 100%; box-sizing: borde
    .checkbox-group { display: flex; align-items: center; gap: 8px; margin-top: 10px;
    button { width: 100%; padding: 10px; background: #0066cc; color: #fff; font-weigh
    button:hover { background: #0052a3; }
  </style>
</head>
<body>
  <div class="form-group">
    <label>Nombre del Módulo:</label>
    <input type="text" id="nombre" value="Librero Simple">
  </div>
  <div class="form-group">
    <label>Alto Total (mm):</label>
    <input type="number" id="alto" value="600" step="1">
  </div>
  <div class="form-group">
    <label>Ancho Total (mm):</label>
    <input type="number" id="ancho" value="500" step="1">
  </div>
  <div class="form-group">
    <label>Profundidad (mm):</label>
    <input type="number" id="profundidad" value="300" step="1">
  </div>
  <div class="form-group">
    <label>Espesor Melamina (mm):</label>
    <select id="espesor">
      <option value="18" selected>18 mm</option>
```

```
        <option value="15">15 mm</option>
      </select>
    </div>
    <div class="checkbox-group">
      <input type="checkbox" id="tiene_repisa" checked>
      <label for="tiene_repisa" style="margin: 0;">Incluir repisa interior</label>
    </div>
    <div class="form-group" style="margin-top: 8px;">
      <label>Altura Repisa desde la base (mm):</label>
      <input type="number" id="altura_repisa" value="282" step="1">
    </div>
    <div class="checkbox-group">
      <input type="checkbox" id="fondo" checked>
      <label for="fondo" style="margin: 0;">Ranura + Fondo MDF 3mm</label>
    </div>

    <button onclick="generarMueble()">Crear Módulo en 3D</button>

    <script>
      function generarMueble() {
        const payload = {
          nombre: document.getElementById('nombre').value,
          alto: parseFloat(document.getElementById('alto').value),
          ancho: parseFloat(document.getElementById('ancho').value),
          profundidad: parseFloat(document.getElementById('profundidad').value),
          espesor: parseFloat(document.getElementById('espesor').value),
          tiene_repisa: document.getElementById('tiene_repisa').checked,
          altura_repisa: parseFloat(document.getElementById('altura_repisa').value),
          fondo: document.getElementById('fondo').checked
        };

        // Comunicación con la API de Ruby en SketchUp
        if (window.sketchup) {
          sketchup.crearModulo(JSON.stringify(payload));
        } else {
          console.log("Datos a enviar a SketchUp:", payload);
        }
      }
    </script>
  </body>
  </html>
```

Siguientes Pasos Clave

1. **Prueba local:** Copia los archivos dentro del directorio de plugins de tu SketchUp y reinicia el programa. En el menú superior aparecerá `Plugins > Abrir MelaStudio Paramétrico`.

2. **Exportador a Excel/CSV:** Como cada pieza creada almacena su diccionario `DespieceMela`, el siguiente módulo lee todas las piezas seleccionadas y genera el archivo CSV formateado con las columnas de corte estándar (`Pieza, Cantidad, Largo, Ancho, Espesor, Veta, Canto Largo 1, Canto Largo 2, Canto Ancho 1, Canto Ancho 2`).

¿Quieres que desarrollemos el script en Ruby para extraer y exportar el CSV/Excel de despiece?

↪ Sí

en sketchup veo su barra de herramientas con varias opciones, tienes el detalle que hace cada opcion

La barra de herramientas más completa y habitual en SketchUp es el **Conjunto grande de herramientas** (*Large Tool Set*). Se organiza en grupos funcionales según la tarea a realizar:

1. Selección y Modificación Básica

- **Seleccionar (Flecha negra / Espacio):** Elige caras, aristas, grupos o componentes para trabajar sobre ellos.
- **Borrador (E):** Elimina líneas y caras al tocarlas.
  - Con **Shift**: Oculta la arista sin borrar la geometría.
  - Con **Ctrl**: Suaviza (*smooth*) aristas curvas.
- **Pintar / Cubo de pintura (B):** Abre el panel de materiales para aplicar colores, vetas de madera o texturas a caras o piezas completas.

2. Dibujo 2D (Líneas y Formas)

- **Línea / Lápiz (L):** Traza líneas rectas punto a punto. Si cierras un contorno plano, genera una cara automáticamente.
- **Mano alzada:** Dibuja líneas curvas irregulares con trazo continuo.
- **Rectángulo (R):** Crea caras rectangulares indicando dos esquinas opuestas (puedes teclear dimensiones separadas por punto y coma, ej. `500;300`).
- **Rectángulo girado:** Dibuja un rectángulo indicando primero un eje de inclinación o dirección y luego el ancho.

- **Círculo (C):** Traza círculos a partir de un centro y un radio (permite definir la cantidad de lados para suavizar la curva).
- **Polígono:** Dibuja figuras regulares de lados (por defecto hexágonos).
- **Arco (2 puntos / 3 puntos / Desde el centro):** Crea segmentos curvos para redondeos de esquinas, chaflanes o molduras.

3. Modelado y Edición 3D

- **Empujar/Tirar (*Push/Pull*) (P):** Extruye una cara plana para darle volumen 3D o perforar un sólido. Es la herramienta central para convertir un plano en una tabla de melamina (ej. tecleando `18`).
- **Sígueme (*Follow Me*):** Extruye una cara 2D a lo largo de una trayectoria o arista. Se usa para molduras, boleados de cantos o ranurados en curvas.
- **Escala (S):** Estira, encoge o deforma un objeto seleccionado mediante tiradores.
- **Equidistancia (*Offset*) (F):** Genera copias paralelas hacia adentro o afuera de los bordes de una cara seleccionada (muy útil para diseñar marcos o divisiones).

4. Transformación y Posicionamiento

- **Mover (M):** Desplaza geometrías o piezas.
  - Presionando **Ctrl**: Clona o duplica el objeto.
  - Presionando **Ctrl** y luego tecleando `5x` o `/3`: Genera matrices lineales de copias múltiples.
- **Rotar (Q):** Gira geometrías alrededor de un transportador según el eje seleccionado (rojo, verde o azul).

5. Medición y Construcción Auxiliar

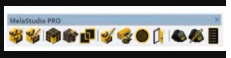
- **Cinta métrica (T):** Mide distancias exactas. Al arrastrar desde una arista, crea **líneas guía discontinuas** indispensables para alinear muebles y marcar ranuras. También escala el modelo completo si mides una distancia conocida y tecleas el valor real.
- **Acotación:** Inserta cotas visibles permanentes (largo, ancho o alto) entre dos puntos.
- **Transportador:** Mide ángulos y traza líneas guía inclinadas.
- **Texto:** Inserta anotaciones vinculadas con una flecha a una cara o arista (muestra automáticamente el área o coordenadas).
- **Texto 3D:** Modela texto extruido con volumen.
- **Ejes:** Reubica o reorienta los ejes de coordenadas locales (, , / rojo, verde, azul).

6. Cámara y Navegación

- **Orbitar (O o presionar la rueda central del ratón):** Rota la vista alrededor del modelo en el espacio 3D.
- **Desplazar / Mano (*Pan*) (H o Shift + rueda del ratón):** Mueve la pantalla lateral o verticalmente sin rotar el ángulo.
- **Zoom (Z o girar la rueda del ratón):** Acerca o aleja la cámara.
- **Ventana de zoom:** Acerca la vista a una zona específica dibujando un recuadro.
- **Ver modelo centrado (*Zoom Extents*) (Ctrl + Shift + E):** Centra y encuadra todo el proyecto visible en pantalla.
- **Situar cámara y Girar:** Coloca la vista a la altura de los ojos humanos para simular un recorrido interior.
- **Caminar:** Permite desplazarse por el modelo con colisiones activadas.

7. Plano de Sección

- **Plano de sección:** Corta visualmente el modelo en un plano transversal para revisar ensamble de cajones, fondos ranurados o estructuras internas sin romper la geometría.



me refiero a esta barra de herramientas

Esta es la barra de herramientas flotante propia de **MelaStudio PRO** para SketchUp. Cada icono corresponde a un módulo especializado del flujo de diseño, fabricación y despiece de melamina:

PNG

Desglose de Funciones (de izquierda a derecha)

1. Módulos y Estructuras Paramétricas

- **Icono 1 (Cubo con piezas separadas): Generador de Módulos Base.** Abre la ventana principal para crear carcasas o estructuras estándar desde cero (gabinetes bajos,

aéreos, columnas, roperos o libreros) indicando alto, ancho, fondo y espesor del tablero.



- **Icono 2 (Cubo con lápiz): Editar / Modificar Módulo Seleccionado.** Permite hacer clic en cualquier mueble creado previamente para reabrir su formulario paramétrico y alterar sus cotas o divisiones sin redibujarlo.



2. Distribución y Espacios Interiores

- **Icono 3 (Cubo segmentado con divisiones amarillas): División Interior / Repisas.** Administra la distribución interna del vano: agregar repisas fijas o móviles, montantes verticales (parantes) y calcular espacios libres (*nichos*) equidistantes de forma automática.



- **Icono 4 (Cubo con módulo secundario interior): Insertar Submódulos / Cajoneras.** Inserta estructuras internas preconfiguradas dentro de un espacio seleccionado (como un cajonero de 3 o 4 cajones con sus holguras de correderas ya descontadas).



3. Ensambles, Ranuras y Acabados

- **Icono 5 (Dos marcos cuadrados superpuestos): Fondos y Ranurado.** Configura la trasera del mueble: define si el fondo de MDF/Trupan (3 mm) va enranurado en los laterales/piso/techo o clavado/atornillado por detrás, aplicando los milímetros de profundidad de ranura.



- **Icono 6 (Panel con lápiz / cota): Asignador de Tapacantos.** Permite seleccionar piezas o cantos específicos para aplicar canto delgado (0.45 mm) o canto grueso (1 mm, 2 mm o 3 mm), aplicando el descuento dimensional correspondiente a la medida neta de corte.



- **Icono 7 (Pieza con pernos / uniones): Mecanizado y Perforaciones (Minifix / Tornillos / Tarugos).** Inserta los puntos de unión mecánica, marcas de broca para tornillo soberbio o perforaciones para herrajes tipo Minifix / Rafix.



- **Icono 8 (Sello / Textura circular): Gestor de Materiales y Vetas.** Asigna el catálogo de melaminas (tableros unicolores o maderados) y define la orientación obligatoria de la veta para que el optimizador no gire la pieza en el corte.



4. Frentes y Accesorios

- **Icono 9 (Puerta semiabierta): Generador de Puertas y Tapas de Cajón.** Crea puertas batientes (interiores o exteriores) y frentes de cajón calculando automáticamente las

luces o holguras perimetrales de 2 mm a 3 mm y la perforación de cazoleta de 35 mm para bisagras.
PNG

5. Identificación y Reportes de Taller

- **Icono 10 (Etiquetas colgantes): Etiquetado de Piezas.** Asigna identificadores únicos (*tags* o stickers) con nombre (ej. `LAT-IZQ`, `PISO`, `TAPA-1`) a cada componente del modelo para el seguimiento en armado y rotulado.
  PNG
- **Icono 11 (Etiqueta con lápiz): Editar Propiedades de Pieza Individual.** Abre un panel para modificar manualmente atributos de una sola tabla (sobredimensionar para refile, cambiar nombre o forzar giro).
  PNG
- **Icono 12 (Tablilla con lista de verificación): Extracción de Despiece / Exportar Lista.** Lee todas las piezas del modelo o selección y genera la tabla consolidada para exportar directamente a **Excel / CSV** formateado para tiendas y optimizadores de corte (Largo, Ancho, Cantidad, Material, Cantos y Ranuras).
  PNG

Cómo estructurar la barra en tu código de Ruby

Para registrar esta misma botonera flotante en SketchUp usas la clase `UI::Toolbar`:

Ruby

```ruby
toolbar = UI::Toolbar.new("MelaStudio Clone")

cmd_crear = UI::Command.new("Crear Módulo") { MelaStudioClone::App.abrir_creador }
cmd_crear.small_icon = "icons/crear_modulo_16.png"
cmd_crear.large_icon = "icons/crear_modulo_24.png"
cmd_crear.tooltip = "Crear nuevo módulo paramétrico"
toolbar.add_item(cmd_crear)

cmd_editar = UI::Command.new("Editar") { MelaStudioClone::App.editar_seleccion }
cmd_editar.small_icon = "icons/editar_16.png"
cmd_editar.large_icon = "icons/editar_24.png"
cmd_editar.tooltip = "Editar módulo seleccionado"
toolbar.add_item(cmd_editar)

toolbar.add_separator

cmd_despiece = UI::Command.new("Exportar Despiece") { MelaStudioClone::Exporter.gener
cmd_despiece.small_icon = "icons/despiece_16.png"
cmd_despiece.large_icon = "icons/despiece_24.png"
```

```
cmd_despiece.tooltip = "Extraer lista de corte a Excel"
toolbar.add_item(cmd_despiece)

toolbar.restore if toolbar.get_last_state == TB_VISIBLE
```